CSE 3204
Formal Language, Automata and Computability

Lecture 16
Deterministic PDA

Deterministic PDA

- A PDA is deterministic if there is never a choice of move in any situation.
- Two types of move.
  - If δ(q, a, X) contains more than one pair then surely the PDA is nondeterministic.
  - Even if δ(q, a, X) is always a singleton, there is a choice between using a real input symbol or ∈

or making a move on ϵ. Thus, we define a PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ to be *deterministic* (a deterministic PDA or DPDA), if and only if the following conditions are met:

1. $\delta(q, a, X)$ has at most one member for any q in Q, a in Σ or $a = \epsilon$, and X in Γ.

2. If $\delta(q, a, X)$ is nonempty, for some a in Σ, then $\delta(q, \epsilon, X)$ must be empty.

Design of a PDA for $L_{wwr} = \{ww^R \mid w \text{ is in } (0+1)^*\}$

$$P = (\{q_0, q_1, q_2\}, \{0, 1\}, \{0, 1, Z_0\}, \delta, q_0, Z_0, \{q_2\})$$

1. $\delta(q_0, 0, Z_0) = \{(q_0, 0Z_0)\}$ and $\delta(q_0, 1, Z_0) = \{(q_0, 1Z_0)\}$.

2. $\delta(q_0, 0, 0) = \{(q_0, 00)\}$, $\delta(q_0, 0, 1) = \{(q_0, 01)\}$, $\delta(q_0, 1, 0) = \{(q_0, 10)\}$, $\delta(q_0, 1, 1) = \{(q_0, 11)\}$.

3. $\delta(q_0, \epsilon, Z_0) = \{(q_1, Z_0)\}$, $\delta(q_0, \epsilon, 0) = \{(q_1, 0)\}$, and $\delta(q_0, \epsilon, 1) = \{(q_1, 1)\}$

4. $\delta(q_1, 0, 0) = \{(q_1, \epsilon)\}$, and $\delta(q_1, 1, 1) = \{(q_1, \epsilon)\}$

5. $\delta(q_1, \epsilon, Z_0) = \{(q_2, Z_0)\}$

Design of a DPDA for $L_{wcwr} = \{wcw^R \mid w \text{ is in } (0+1)^*\}$

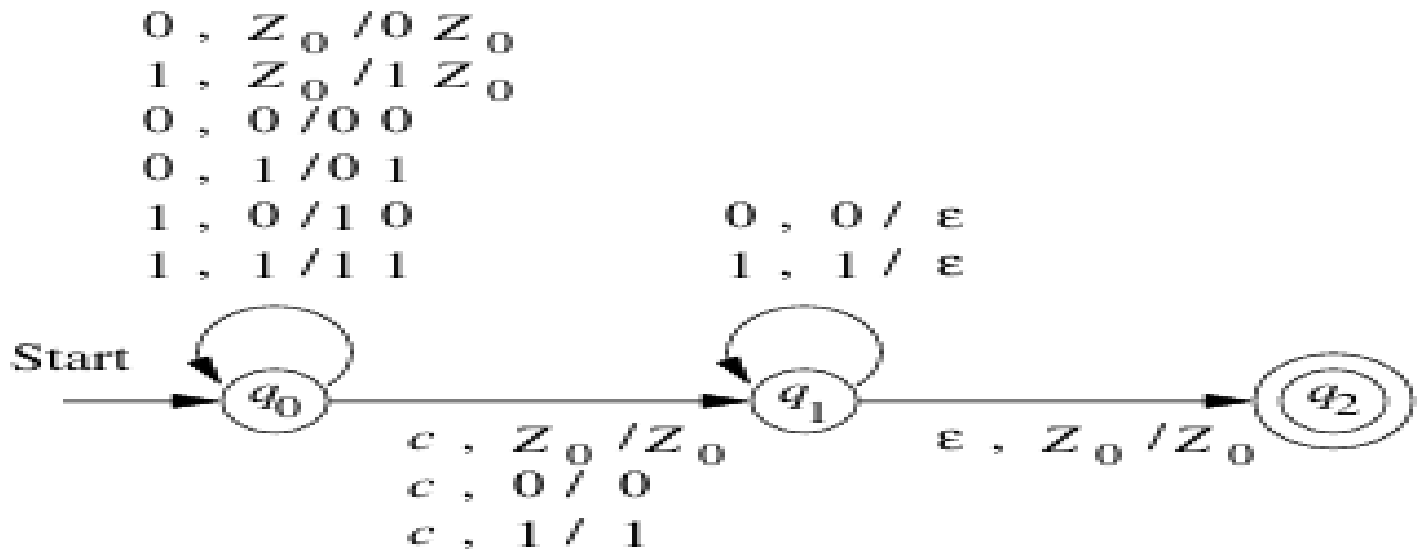

Figure 6.11: A deterministic PDA accepting L_{wcwr}

Regular languages and DPDA

Theorem 6.17: If L is a regular language, then $L = L(P)$ for some DPDA P.

PROOF: Essentially, a DPDA can simulate a deterministic finite automaton. The PDA keeps some stack symbol Z_0 on its stack, because a PDA has to have a stack, but really the PDA ignores its stack and just uses its state. Formally, let $A = (Q, \Sigma, \delta_A, q_0, F)$ be a DFA. Construct DPDA

$$P = (Q, \Sigma, \{Z_0\}, \delta_P, q_0, Z_0, F)$$

by defining $\delta_P(q, a, Z_0) = \{(p, Z_0)\}$ for all states p and q in Q, such that $\delta_A(q, a) = p$.

We claim that $(q_0, w, Z_0) \overset{*}{\underset{P}{\vdash}} (p, \epsilon, Z_0)$ if and only if $\hat{\delta}_A(q_0, w) = p$. That is, P simulates A using its state. The proofs in both directions are easy inductions

Theorem 6.19: A language L is $N(P)$ for some DPDA P if and only if L has the prefix property and L is $L(P')$ for some DPDA P'. □

Regular languages and DPDA

- Prefix Property:

A language L has the prefix property if there are no two different strings x and y in L such that x is a prefix of y.

- Suppose x and xy in L and x is a prefix of y.
  In deterministic PDA by empty stack, the stack must be empty when x is read.
  For xy, we need to continue again after emptying the stack by reading x, which is not permissible.

Example 6.18 : The language L_{wcwr} of Example 6.16 has the prefix property. That is, it is not possible for there to be two strings wcw^R and xcx^R, one of which is a prefix of the other, unless they are the same string. To see why, suppose wcw^R is a prefix of xcx^R, and $w \neq x$. Then w must be shorter than x. Therefore, the c in wcw^R comes in a position where xcx^R has a 0 or 1; it is a position in the first x. That point contradicts the assumption that wcw^R is a prefix of xcx^R.

- $\{0\}^*$ is a regular language and does not have prefix property and therefore not accepted by DPDA by empty stack.

Normal Forms for Context-Free Grammars

- Every CFL (without $\in$) is generated by a CFG in which all productions are of the form A → BC or A→ a where A, B and C are variables and a is a terminal This form is called Chomsky Normal Form.
- To get in there,

1. We must eliminate *useless symbols*, those variables or terminals that do not appear in any derivation of a terminal string from the start symbol.
2. We must eliminate *ϵ-productions*, those of the form $A \to \epsilon$ for some variable A.
3. We must eliminate *unit productions*, those of the form $A \to B$ for variables A and B.

Eliminating Useless Symbols

We say a symbol X is *useful* for a grammar $G = (V, T, P, S)$ if there is some derivation of the form $S \stackrel{*}{\Rightarrow} \alpha X \beta \stackrel{*}{\Rightarrow} w$, where w is in T^*. Note that X may be in either V or T, and the sentential form $\alpha X \beta$ might be the first or last in the

Omitting useless symbols does not change the generated language L.

Our approach to eliminating useless symbols begins by identifying the two things a symbol has to be able to do to be useful:

1. We say X is *generating* if $X \stackrel{*}{\Rightarrow} w$ for some terminal string w. Note that every terminal is generating, since w can be that terminal itself, which is derived by zero steps.

2. We say X is *reachable* if there is a derivation $S \stackrel{*}{\Rightarrow} \alpha X \beta$ for some α and β.

Eliminating Useless Symbols

Example 7.1: Consider the grammar:

$$S \to AB \mid a$$
$$A \to b$$

All symbols but B are generating; a and b generate themselves; S generates a, and A generates b. If we eliminate B, we must eliminate the production $S \to AB$, leaving the grammar:

$$S \to a$$
$$A \to b$$

Now, we find that only S and a are reachable from S. Eliminating A and b leaves only the production $S \to a$. That production by itself is a grammar whose language is $\{a\}$, just as is the language of the original grammar.

Note that if we start by checking for reachability first, we find that all symbols of the grammar

$$S \to AB \mid a$$
$$A \to b$$

are reachable. If we then eliminate the symbol B because it is not generating, we are left with a grammar that still has useless symbols, in particular, A and b. □

Computing the Generating and Reachable Symbols

Let $G = (V, T, P, S)$ be a grammar. To compute the generating symbols of G, we perform the following induction.

BASIS: Every symbol of T is obviously generating; it generates itself.

INDUCTION: Suppose there is a production $A \to \alpha$, and every symbol of α is already known to be generating. Then A is generating. Note that this rule includes the case where $\alpha = \epsilon$; all variables that have ϵ as a production body are surely generating.

Reachable Symbols:

BASIS: S is surely reachable.

INDUCTION: Suppose we have discovered that some variable A is reachable. Then for all productions with A in the head, all the symbols of the bodies of those productions are also reachable.

Theorem 7.4: The algorithm above finds all and only the generating symbols of G.

PROOF: For one direction, it is an easy induction on the order in which symbols are added to the set of generating symbols that each symbol added really is generating. We leave to the reader this part of the proof.

For the other direction, suppose X is a generating symbol, say $X \overset{*}{\underset{G}{\Rightarrow}} w$. We prove by induction on the length of this derivation that X is found to be generating.

BASIS: Zero steps. Then X is a terminal, and X is found in the basis.

INDUCTION: If the derivation takes n steps for $n > 0$, then X is a variable. Let the derivation be $X \Rightarrow \alpha \overset{*}{\Rightarrow} w$; that is, the first production used is $X \to \alpha$. Each symbol of α derives some terminal string that is a part of w, and that derivation must take fewer than n steps. By the inductive hypothesis, each symbol of α is found to be generating. The inductive part of the algorithm allows us to use production $X \to \alpha$ to infer that X is generating. □

Eliminating ∈-Productions

- Nullable Variable: A variable A is nullable if $A \stackrel{*}{\Rightarrow} \epsilon$.

Our strategy is to begin by discovering which variables are "nullable." A variable A is *nullable* if $A \stackrel{*}{\Rightarrow} \epsilon$. If A is nullable, then whenever A appears in a production body, say $B \to CAD$, A might (or might not) derive ϵ. We make two versions of the production, one without A in the body ($B \to CD$), which corresponds to the case where A would have been used to derive ϵ, and the other with A still present ($B \to CAD$). However, if we use the version with A present, then we cannot allow A to derive ϵ. That proves not to be a problem, since we shall simply eliminate all productions with ϵ bodies, thus preventing any variable from deriving ϵ.

$$L(G_1) = L(G) - \{\epsilon\}$$

Let $G = (V, T, P, S)$ be a CFG. We can find all the nullable symbols of G by the following iterative algorithm. We shall then show that there are no nullable symbols except what the algorithm finds.

BASIS: If $A \to \epsilon$ is a production of G, then A is nullable.

INDUCTION: If there is a production $B \to C_1C_2 \cdots C_k$, where each C_i is nullable, then B is nullable. Note that each C_i must be a variable to be nullable, so we only have to consider productions with all-variable bodies.

Now we give the construction of a grammar without ϵ-productions. Let $G = (V, T, P, S)$ be a CFG. Determine all the nullable symbols of G. We construct a new grammar $G_1 = (V, T, P_1, S)$, whose set of productions P_1 is determined as follows.

For each production $A \to X_1X_2 \cdots X_k$ of P, where $k \geq 1$, suppose that m of the k X_i's are nullable symbols. The new grammar G_1 will have 2^m versions of this production, where the nullable X_i's, in all possible combinations are present or absent. There is one exception: if $m = k$, i.e., all symbols are nullable, then we do not include the case where all X_i's are absent. Also, note that if a production of the form $A \to \epsilon$ is in P, we do not place this production in P_1.

Eliminating ∈-Productions

$S \rightarrow AB$
$A \rightarrow aAA \mid \epsilon$
$B \rightarrow bBB \mid \epsilon$

Set of nullable symbols = {S, A, B}

$S \rightarrow AB.$	$S \rightarrow AB \mid A \mid B$
$A \rightarrow aAA.$	$A \rightarrow aAA \mid aA \mid aA \mid a$
$B \rightarrow bBB$	$B \rightarrow bBB \mid bB \mid b$

Eliminating Unit Productions

- A unit production is a production of the form A→B where both A and B are variables.
- Can create extra steps in derivation.

$$
\begin{array}{lcl}
I & \rightarrow & a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \\
F & \rightarrow & I \mid (E) \\
T & \rightarrow & F \mid T * F \\
E & \rightarrow & T \mid E + T
\end{array}
$$

$$E \rightarrow T \rightarrow E \rightarrow F \mid T*F \rightarrow E \rightarrow I \mid (E) \mid T*F \rightarrow E \rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \mid (E) \mid T*F \mid E+T$$

The technique suggested above — expand unit productions until they disappear — often works. However, it can fail if there is a cycle of unit productions, such as $A \rightarrow B$, $B \rightarrow C$, and $C \rightarrow A$. The technique that is guaranteed to work involves first finding all those pairs of variables A and B such that $A \stackrel{*}{\Rightarrow} B$ using a sequence of unit productions only. Note that it is possible for $A \stackrel{*}{\Rightarrow} B$ to be true even though no unit productions are involved. For instance, we might have productions $A \rightarrow BC$ and $C \rightarrow \epsilon$.

Eliminating Unit Productions

Once we have determined all such pairs, we can replace any sequence of derivation steps in which $A \Rightarrow B_1 \Rightarrow B_2 \Rightarrow \cdots \Rightarrow B_n \Rightarrow \alpha$ by a production that uses the nonunit production $B_n \rightarrow \alpha$ directly from A; that is, $A \rightarrow \alpha$. To begin, here is the inductive construction of the pairs (A, B) such that $A \stackrel{*}{\Rightarrow} B$ using only unit productions. Call such a pair a *unit pair*.

BASIS: (A, A) is a unit pair for any variable A. That is, $A \stackrel{*}{\Rightarrow} A$ by zero steps.

INDUCTION: Suppose we have determined that (A, B) is a unit pair, and $B \rightarrow C$ is a production, where C is a variable. Then (A, C) is a unit pair.

$$
\begin{array}{lcl}
I & \rightarrow & a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \\
F & \rightarrow & I \mid (E) \\
T & \rightarrow & F \mid T * F \\
E & \rightarrow & T \mid E + T
\end{array}
$$

Example 7.10 : Consider the expression grammar of Example 5.27, which we reproduced above. The basis gives us the unit pairs (E, E), (T, T), (F, F), and (I, I). For the inductive step, we can make the following inferences:

1. (E, E) and the production $E \rightarrow T$ gives us unit pair (E, T).
2. (E, T) and the production $T \rightarrow F$ gives us unit pair (E, F).
3. (E, F) and the production $F \rightarrow I$ gives us unit pair (E, I).
4. (T, T) and the production $T \rightarrow F$ gives us unit pair (T, F).
5. (T, F) and the production $F \rightarrow I$ gives us unit pair (T, I).
6. (F, F) and the production $F \rightarrow I$ gives us unit pair (F, I).

Eliminating Unit Productions

To eliminate unit productions, we proceed as follows. Given a CFG $G = (V, T, P, S)$, construct CFG $G_1 = (V, T, P_1, S)$:

1. Find all the unit pairs of G.
2. For each unit pair (A, B), add to P_1 all the productions $A \to \alpha$, where $B \to \alpha$ is a nonunit production in P. Note that $A = B$ is possible; in that way, P_1 contains all the nonunit productions in P.

Pair	Productions
(E, E)	$E \to E + T$
(E, T)	$E \to T * F$
(E, F)	$E \to (E)$
(E, I)	$E \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$
(T, T)	$T \to T * F$
(T, F)	$T \to (E)$
(T, I)	$T \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$
(F, F)	$F \to (E)$
(F, I)	$F \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$
(I, I)	$I \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$

$E \to E + T \mid T * F \mid (E) \mid a \mid b \mid Ia \mid Ib \mid I0 \mid I1$
$T \to T * F \mid (E) \mid a \mid b \mid Ia \mid Ib \mid I0 \mid I1$
$F \to (E) \mid a \mid b \mid Ia \mid Ib \mid I0 \mid I1$
$I \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$

Figure 7.1: Grammar constructed by step (2) of the unit-production-elimination algorithm

- Section 6.4 of Chapter 6
- Section 7.1 of Chapter 7